

Variable Scoping - Python

Scope: Region, where a Variable is defined

- **Local Scope:** Variables defined inside a function.
- **Enclosing Scope:** Variables in the outer function of a nested function.
- **Global Scope:** Variables defined outside all functions.
- **Built-in Scope:** Reserved keywords and functions provided by Python.

Python Order
LEGB-Rule

- Python looks for a variable in order
- This allows users to overwrite built-in functions

```
x = 20
def my_function():
    global x
    x = 30 # Modifies the global variable
my_function()
print(x) # Output: 30

def outer_function():
    x = 25 # Enclosing variable
    def inner_function():
        nonlocal x
        x = 30 # Modifies the enclosing variable
    inner_function()
outer_function() # Output: 30
```

Call Stack - Python

- LiFo data structure
- push
- pop
- peek → look at top elmt

Call Stack manages function calls in Python

- Call Stack Flow
- 1. func1() is called → Pushed onto the stack.
- 2. func1() calls func2() → func2() is pushed onto the stack.
- 3. func2() completes → Popped from the stack.
- 4. func1() completes → Popped from the stack.

- CS manages nested or recursive function calls
- ↳ Stack grows to large → StackOverflow Error

Dynamic Dispatch

- To find a function or property of an Object while the program is running

obj.name → obj["name"]

Snapshot - File Archive

- For every Backup all current files are copied to a new Folder
- Wasteful

Manifest - File Archive

- Only ce-safe files that are new or that have changed
- The "Manifests" keep track of where the files of each Backup will be
- When doing a Backup → check manifest if a file has to be "re-saved"
- Deterministic hash functions are used to ensure that the content of files hasn't changed

Lexical Scoping

- Variable refers to top level env
- Variables are resolved on where these outer function is **defined** not from where its **called**
- Standard in Python

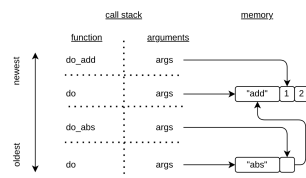
Dynamic Scoping

- Variables are define dynamically by current environment

Interpreter

- Parser: function to transcribe sourcecode into data structure
- Runtime: Executes instructions in the data structure

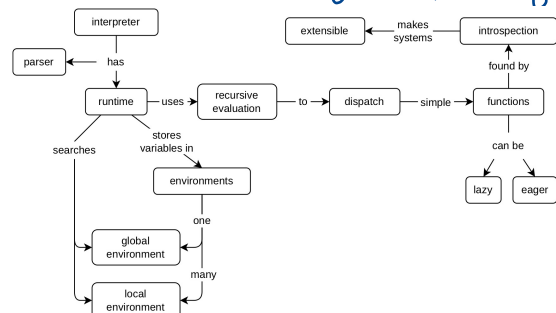
- Compiler: Translates before running (≈ Java)
- Interpreter: Translates on the fly (Python)



→ Recursion through dynamic Dispatch

Recursively evaluating the expression ['abs', ['add', 1, 2]].

- Control Flow: Sequence of instructions order
- Expression: Instruction that produces output
- Statement: Instruction that does not produce output
- Introspection: Reflect on programs properties during run



Finding Duplicates

```
def find_duplicates(filename):
    matches = []
    for left in filenames:
        for right in filenames:
            if same_bytes(left, right):
                matches.append((left, right))
    return matches

def same_bytes(left_name, right_name):
    left_bytes = open(left_name, "rb").read()
    right_bytes = open(right_name, "rb").read()
    return left_bytes == right_bytes
```

Slow $O(N^2)$

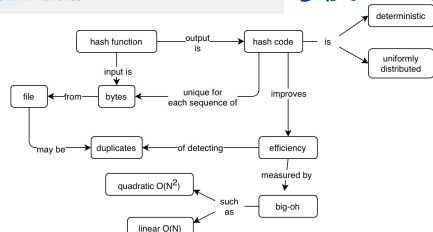


Figure 3.5: Concept map for duplicate file detection with hashing.

Functions and Closure

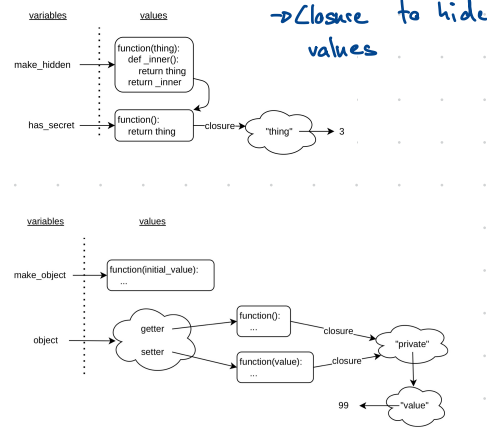
→ Eager Eval: Args are evaluated before functions are called

→ Lazy Eval: Just-in-time evaluation of Args

→ Anonymous Func: Func without Name
↳ in Python "Lambda func"

→ Call Stack: List of dicts off all active processes

→ Stack Frame: Subdict of single process



Protocols

→ Mock Objects: to simplify testing with e.g. fakes

```
def test_with_real_function():
    assert adder(2, 3) == 5

def test_with_fixed_return_value():
    fake = adder
    fake.__name__ = 'value'
    assert adder(2, 3) == 99
```

adder

→ Built in "fake" function in Python is with func as name:
↳ runs enter → func → exit
↳ Built in instructions like that are call protocols

Decorators

SoCo 24

→ One of python's protocols → wraps a func

```
def wrap(func):
    def inner(*args):
        print("before call")
        func(*args)
        print("after call")
    return inner
```

before call
original: example
after call

```
@wrap
def original(message):
    print(f"original: {message}")

original("example")
```

→ To be able to pass arguments as we apply the decorator, we would have yet another nested function to the mix

Atomic Operation

→ Operation that can not be interrupted pathway through

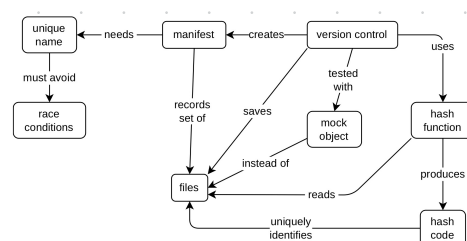


Figure 10.3: Concept map for hashing-based file backup.

Iterators

→ Another python protocol

```
class NaiveIterator:
    def __init__(self, text):
        self._text = text[:]

    def __iter__(self):
        self._row, self._col = 0, -1
        return self

    def __next__(self):
        self._advance()
        if self._row == len(self._text):
            raise StopIteration
        return self._text[self._row][self._col]
```

→ to move through an object in a specialized fashion

```
def gather(buffer):
    result = ""
    for char in buffer:
        result += char
    return result
```

```
def test_naive_buffer():
    buffer = NaiveIterator(["ab", "c"])
    assert gather(buffer) == "abc"
```

Object Oriented Coding

→ Polymorphism: multiple Classes have the same methods:

Circle.area() \ Square.area()

→ Aliasing: Assigning a object to a new name

→ Inheritance: Creating New Class from existing Class

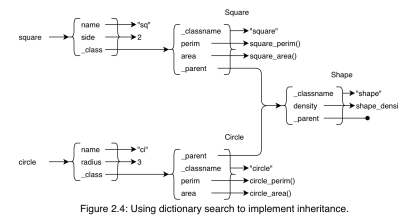


Figure 2.4: Using dictionary search to implement inheritance.

→ Multiple Inheritance: Creatin New from multiple Parents

→ Monkey Patching: Replace Methods of a class during Runtime

→ Varsargs: Variable arguments *args / **kwargs

→ Spreading: Using star operator to spread a list

```
def show_spread(left, middle, right):
    print(f"left {left} middle {middle} right {right}")
```

```
all_in_list = [1, 2, 3]
show_spread(*all_in_list)
```

```
all_in_dict = {"right": 30, "left": 10, "middle": 20}
show_spread(**all_in_dict)
```

```
left 1 middle 2 right 3
left 10 middle 20 right 30
```

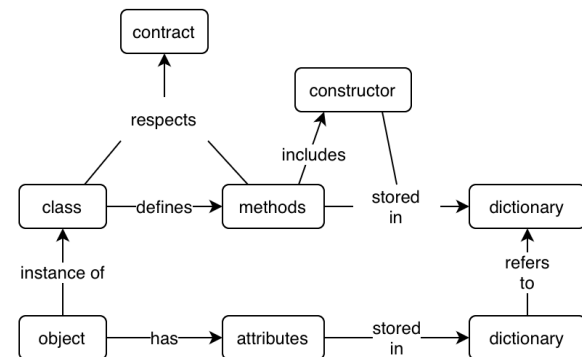
spread.py

→ Constructor: Function that create Object of specific Class

→ Vpcall: Calling a "parent-method" from a child location

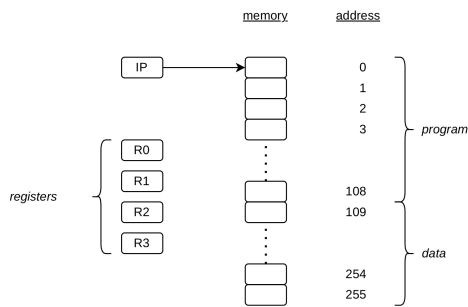
→ Class Method: Method of Class that takes the class object as an input

→ Static Method: Method of Class that does not rely on object of class or class itself



Virtual Machine

- Instruction Pointer (IP) : Holds Memory Address of next instruction
- Application Binary Interface (ABI) : Lowlevel Layout Software needs to run
 - ↳ IP is set to zero in beginning
- Application Programming Interface (API) : Set of provided base functions.
- Word : Piece of Memory of size which hardware likes
- Register : Memory Location of one word



- Instruction Set : Base operands which the chip can do
- Assembly Code : Base operands in human readable form

Name	Code	Format	Action	Example	Equivalent
hlt	1	--	Halt program	hlt	sys.exit(0)
ldc	2	rv	Load constant	ldc R0 99	R0 = 99
ldr	3	rr	Load register	ldr R0 R1	R0 = memory[R1]
cpy	4	rr	Copy register	cpy R0 R1	R0 = R1
str	5	rr	Store register	str R0 R1	memory[R1] = R0
add	6	rr	Add	add R0 R1	R0 = R0 + R1
sub	7	rr	Subtract	sub R0 R1	R0 = R0 - R1
beq	8	rv	Branch if equal	beq R0 99	if (R0==0) IP = 99
bne	9	rv	Branch if not equal	bne R0 99	if (R0!=0) IP = 99
prp	10	r-	Print register	prp R0	print(R0)
prm	11	r-	Print memory	prm R0	print(memory[R0])

=>> Bitshift to right 0110 ⁶ → 0011 ³
<<= Bitshift to left
|= Bitwise "or" 0110 |= 1001 → 1111

